

DP-900 Azure Data Fundamentals - Exam Prep

Skills Assessment

- Describe core data concepts (15-20%)
- Describe how to work with relational data on Azure (25-30%)
- Describe how to work with non-relational data on Azure (25-30%)
- Describe an analytics workload on Azure (25-30%)

Table of Contents

1. [Explore Job Roles in the World of Data](#)
2. [Identify Data Formats](#)
3. [Explore File Storage](#)
4. [Explore Databases](#)
5. [Explore Transactional Data Processing](#)
6. [Explore Analytical Data Processing](#)
7. [Identify Data Services](#)
8. [Understand Relational Data](#)
9. [Understand Normalization](#)
10. [Explore SQL](#)
11. [Describe Database Objects](#)
12. [Azure SQL Services and Capabilities](#)
13. [Azure SQL Services for Open-Source Databases](#)
14. [Explore Azure Blob Storage](#)
15. [Explore Azure Data Lake Storage Gen2](#)
16. [Explore Azure Files](#)
17. [Explore Azure Tables](#)
18. [Azure Cosmos DB](#)
19. [Azure Cosmos DB APIs](#)
20. [Data Warehousing Architecture](#)
21. [Data Ingestion Pipelines](#)
22. [Analytical Data Stores](#)
23. [Analytical Platform-as-a-Service \(PaaS\) Solutions](#)
24. [Microsoft Fabric](#)
25. [Batch and Stream Processing](#)
26. [Common Elements of Stream Processing Architecture](#)
27. [Azure Stream Analytics](#)
28. [Apache Spark on Microsoft Azure](#)
29. [Real-time Analytics in Microsoft Fabric](#)
30. [Power BI Tools and Workflow](#)
31. [Power BI Core Concepts of Data Modeling](#)
32. [Power BI for Data Visualization](#)

Explore Job Roles in the World of Data

Overview

There is a wide variety of roles involved in managing, controlling, and using data. Roles can be business-oriented, engineering-focused, research-based, or hybrid. While job titles and responsibilities may vary across organizations, the following roles encapsulate common divisions of tasks and responsibilities.

Key Job Roles

Database Administrator (DBA)

- **Responsibilities:**
 - Design, implement, maintain, and manage on-premises and cloud-based databases.
 - Ensure overall availability, performance, and optimization of databases.

- Implement backup and recovery plans to handle failures and disasters.
- Manage data security, grant privileges, and control user access.

Data Engineer

- **Responsibilities:**
 - Collaborate with stakeholders to design and implement data workloads.
 - Develop data ingestion pipelines, cleansing, and transformation activities.
 - Manage data stores for analytical workloads.
 - Ensure data privacy across on-premises and cloud data stores.
 - Monitor data pipelines to ensure expected data loads.

Data Analyst

- **Responsibilities:**
 - Explore data to identify trends and relationships.
 - Design and build analytical models.
 - Create reports and visualizations to enable advanced analytics capabilities.
 - Transform raw data into relevant insights based on business requirements.

Additional Notes

- **Hybrid Roles:** In some organizations, a single person may perform multiple roles. For example, a DBA might also create data pipelines as a Data Engineer.
- **Other Roles:** Additional data-related roles not detailed here include Data Scientist and Data Architect. Technical professionals such as application developers and software engineers also work with data.

Identify Data Formats

Overview

Data is a collection of facts used to record information, often organized in structures representing entities important to an organization (e.g., customers, products). Entities have attributes (e.g., name, address).

Data Classifications

Data can be classified as:

- **Structured**
- **Semi-structured**
- **Unstructured**

Structured Data

- Adheres to a fixed schema.
- Commonly tabular (rows and columns).
- Stored in databases using a relational model.

Example: Customer and Product tables

Semi-structured Data

- Has some structure with variations between instances.
- Common format: JSON.

Example:

- Customer documents with varying address and contact information.

Note: JSON is just one of many semi-structured data representations.

Unstructured Data

- No specific structure.
- Includes documents, images, audio, video, and binary files.

Data Stores

Organizations store data in structured, semi-structured, or unstructured formats for recording details of entities, events, or other information. Stored data is retrieved for analysis and reporting.

Types of Data Stores

1. **File stores**
2. **Databases**

Explore File Storage

Overview

The ability to store data in files is a core element of any computing system. Files can be stored in local file systems, on removable media, or centrally in a shared file storage system, increasingly hosted in the cloud for cost-effective, secure, and reliable storage.

Factors Influencing File Format Choice

- Type of data (structured, semi-structured, or unstructured)
- Applications and services needing access to the data
- Human readability vs. optimization for storage and processing

Common File Formats

Delimited Text Files

- **Format:** Plain text with specific field delimiters and row terminators (e.g., CSV, TSV, space-delimited, fixed-width)
- **Usage:** Structured data accessed by various applications and services in a human-readable format.

JavaScript Object Notation (JSON)

- **Format:** Hierarchical document schema defining data entities (objects) with multiple attributes.
- **Usage:** Both structured and semi-structured data, flexible for varying attributes.

Extensible Markup Language (XML)

- **Format:** Human-readable, tag-based.
- **Usage:** Previously popular for structured data, largely replaced by JSON.

Binary Large Object (BLOB)

- **Format:** Raw binary data, interpreted by applications and rendered.
- **Usage:** Unstructured data such as images, video, audio, and application-specific documents.

Optimized File Formats

Specialized formats developed for compression, indexing, and efficient storage and processing.

Avro

- **Type:** Row-based
- **Usage:** Compressing data, minimizing storage and network bandwidth.
- **Features:**

- **Schema Evolution:** Supports schema changes (e.g., adding new fields) without breaking existing data.
- **Data Serialization:** Efficiently serializes data for transport and storage.
- **Interoperability:** Used in data serialization frameworks like Apache Kafka.
- **Header Information:** Each record contains a header stored as JSON that describes the structure of the data, aiding in parsing binary data.
- **Integration:** Well-integrated with Hadoop ecosystems and big data tools.

ORC (Optimized Row Columnar)

- **Type:** Columnar
- **Usage:** Optimizing read and write operations in Apache Hive.
- **Features:**
 - **Columnar Storage:** Organizes data into columns rather than rows, improving query performance.
 - **Stripes:** Divides data into large stripes, each containing an index, data for rows, and a footer with statistical information.
 - **Compression:** Supports lightweight and fast compression techniques like Zlib, LZO, and Snappy.
 - **Predicate Pushdown:** Enhances query performance by allowing queries to skip non-relevant data.
 - **Complex Types:** Efficiently handles complex data types and nested structures.

Parquet

- **Type:** Columnar
- **Usage:** Efficient storage and processing of nested data types.
- **Features:**
 - **Row Groups:** Data is divided into row groups. Each row group contains data for a certain number of rows.
 - **Column Chunks:** Within each row group, data is stored in column chunks. Each column chunk contains data for a single column across multiple rows.
 - **Metadata:** Includes rich metadata that describes the structure of the data, allowing for efficient data retrieval.
 - **Compression:** Supports efficient compression and encoding schemes, reducing storage space and improving read/write performance.
 - **Nested Data Structures:** Optimized for complex nested data structures, making it suitable for big data processing frameworks like Apache Spark and Hadoop.

- **Data Encoding:** Utilizes different encoding techniques for different data types to enhance storage efficiency.

Explore Databases

Overview

A database is a central system used to store and query data records. Unlike file systems, databases are dedicated to managing data records efficiently.

Relational Databases

- **Usage:** Store and query structured data.
- **Structure:** Data is stored in tables representing entities (e.g., customers, products, sales orders).
- **Primary Keys:** Unique identifiers for each entity instance, used to reference entities in other tables.
- **Normalization:** Elimination of duplicate data values, ensuring data is stored only once.
- **Query Language:** Managed and queried using Structured Query Language (SQL), based on ANSI standard.

Example: A customer's primary key referenced in a sales order to indicate which customer placed the order.

Key Features

- **Tables:** Represent entities.
- **Primary Keys:** Unique identifiers.
- **Normalization:** Reduces duplicate data.
- **SQL:** Standard query language.

Non-relational Databases

Non-relational databases, also known as NoSQL databases, do not use a relational schema. They are categorized into four common types:

Key-value Databases

- **Structure:** Each record consists of a unique key and an associated value.
- **Flexibility:** Value can be in any format.

Example: Key-value pairs where the value could be text, JSON, or any other format.

Document Databases

- **Structure:** A form of key-value database where the value is a JSON document.
- **Optimization:** Optimized for parsing and querying JSON documents.

Example: Storing customer information as JSON documents.

Column Family Databases

- **Structure:** Store tabular data with rows and columns.
- **Column Families:** Columns are grouped into families that are logically related.

Example: A database storing user profiles, with column families for personal info and activity logs.

Graph Databases

- **Structure:** Store entities as nodes with links (edges) defining relationships between them.
- **Usage:** Ideal for complex relationships and interconnected data.

Example: A social network database where users are nodes and friendships are edges.

Explore Transactional Data Processing

Overview

A transactional data processing system records transactions, which are discrete units of work encapsulating specific events an organization wants to track. Transactions can be financial, retail, or other types of business activities.

Characteristics of Transactional Systems

- **High Volume:** Often handle millions of transactions per day.
- **Speed:** Data must be quickly accessible.
- **Transactional Processing:** Known as Online Transactional Processing (OLTP).

OLTP Solutions

OLTP solutions use a database system optimized for both read and write operations to support transactional workloads involving CRUD operations (Create, Retrieve, Update, Delete). These operations must ensure data integrity through ACID semantics:

ACID Semantics

- **Atomicity:** Each transaction is a single unit that either fully succeeds or fully fails.
 - Example: A funds transfer transaction must debit one account and credit another completely or not at all.
- **Consistency:** Transactions take the database from one valid state to another.
 - Example: A completed funds transfer must accurately reflect in both accounts involved.
- **Isolation:** Concurrent transactions do not interfere with each other.
 - Example: While a funds transfer is in process, other transactions must return consistent account balances.
- **Durability:** Committed transactions are permanently recorded.
 - Example: Once a funds transfer is committed, it remains recorded even if the system is turned off.

Usage of OLTP Systems

OLTP systems support live applications that process business data, often referred to as line of business (LOB) applications.

Explore Analytical Data Processing

Overview

Analytical data processing uses read-only (or read-mostly) systems to store vast volumes of historical data or business metrics. Analytics can be based on snapshots of data at given points in time or a series of snapshots.

Common Architecture for Enterprise-Scale Analytics

1. **ETL Process:** Operational data is extracted, transformed, and loaded into a data lake for analysis.
2. **Data Loading:** Data is loaded into a schema of tables, typically in a Spark-based data lakehouse or a data warehouse.
3. **Aggregation:** Data in the data warehouse is aggregated and loaded into an OLAP model or

cube. Measures from fact tables are calculated for intersections of dimensions from dimension tables.

4. **Querying:** The data in the data lake, data warehouse, and analytical model can be queried to produce reports, visualizations, and dashboards.

Key Components

Data Lakes

- **Usage:** Collect and analyze large volumes of file-based data.
- **Scalability:** Flexible and scalable storage for large-scale data.

Data Warehouses

- **Usage:** Store data in a relational schema optimized for read operations.
- **Optimization:** Primarily supports queries for reporting and data visualization.

Data Lakehouses

- **Usage:** Combine the flexible storage of a data lake with the relational querying of a data warehouse.
- **Innovation:** Newer approach integrating the best of data lakes and data warehouses.

OLAP Model

- **Usage:** Optimized for analytical workloads.
- **Aggregation:** Pre-aggregated data across dimensions at different levels, enabling drill-up/down functionality.
- **Performance:** Queries return summaries quickly due to pre-aggregation.

Roles in Data Analytical Work

- **Data Scientists:** Work directly with data files in a data lake to explore and model data.
- **Data Analysts:** Query tables in the data warehouse to produce complex reports and visualizations.
- **Business Users:** Consume pre-aggregated data in analytical models via reports or dashboards.

Identify Data Services

Overview

Microsoft Azure is a cloud platform supporting various data services for transactional and analytical workloads. This guide covers some of the most commonly used data services.

Commonly Used Data Services

Azure SQL

- **Services:**
 - **Azure SQL Database:** Fully managed PaaS database.
 - **Azure SQL Managed Instance:** Hosted SQL Server instance with automated maintenance.
 - **Azure SQL VM:** Virtual machine with SQL Server for maximum configurability.
- **Usage:** Support LOB applications, data sources for ETL operations, direct querying for reports.

Azure Database for Open-Source Relational Databases

- **Services:**

- **Azure Database for MySQL**
- **Azure Database for MariaDB**
- **Azure Database for PostgreSQL**

- **Usage:** Support transactional applications, data sources for ETL operations, direct querying for reports.

Azure Cosmos DB

- **Type:** Global-scale NoSQL database.
- **APIs Supported:** JSON documents, key-value pairs, column-families, graphs.
- **Usage:** Managed by database administrators or software developers, integrated into enterprise analytical solutions.

Azure Storage

- **Storage Options:**
 - **Blob Containers:** Scalable, cost-effective storage for binary files.
 - **File Shares:** Network file shares.
 - **Tables:** Key-value storage for quick data access.
- **Usage:** Hosting data lakes with hierarchical namespaces for organized data storage.

Azure Data Factory

- **Function:** Define and schedule data pipelines for data transfer and transformation.
- **Usage:** Build ETL solutions to populate analytical data stores from transactional systems.

Azure Synapse Analytics

- **Features:**
 - **Pipelines:** Similar to Azure Data Factory.
 - **SQL:** Scalable SQL database engine for data warehousing.
 - **Apache Spark:** Distributed data processing system.
 - **Data Explorer:** High-performance querying of log and telemetry data.
- **Usage:** Unified data analytics solution combining data ingestion, data warehouse, and data lake storage.

Azure Databricks

- **Integration:** Azure-integrated Databricks platform combining Apache Spark and SQL database semantics.
- **Usage:** Large-scale data analytics, interactive notebooks for querying and visualizing data.

Azure HDInsight

- **Supported Technologies:**
 - **Apache Spark:** A distributed data processing system that supports multiple programming languages and APIs, including Java, Scala, Python, and SQL.
 - **Apache Hadoop:** A distributed system that uses MapReduce jobs to process large volumes of data efficiently across multiple cluster nodes. MapReduce jobs can be written in Java or abstracted by interfaces such as Apache Hive - a SQL-based API that runs on Hadoop.

- **Apache HBase:** An open-source system for large-scale NoSQL data storage and querying.
- **Apache Kafka:** A message broker for data stream processing.
- **Usage:** Support big data analytics workloads with multiple open-source technologies.

Azure Stream Analytics

- **Function:** Real-time stream processing engine.
- **Usage:** Capture streaming data for ingestion into analytical data stores or real-time visualization.

Azure Data Explorer

- **Function:** High-performance querying of log and telemetry data.
- **Usage:** Query and analyze timestamped data such as log files and IoT telemetry data.

Microsoft Purview

- **Function:** Enterprise-wide data governance and discoverability.
- **Usage:** Create data maps, track data lineage, enforce data governance, ensure data integrity for analytical workloads.

Microsoft Fabric

- **Function:** Unified SaaS analytics platform.
- **Features:**
 - Data ingestion and ETL
 - Data lakehouse analytics
 - Data warehouse analytics
 - Data science and machine learning
 - Real-time analytics
 - Data visualization
 - Data governance and management

Understand Relational Data

Overview

In a relational database, collections of entities from the real world are modeled as tables. An entity can be anything for which you want to record information, typically important objects and events.

Key Concepts

Entities and Tables

- **Entities:** Represent real-world objects or events (e.g., customers, products, orders).
- **Tables:** Model collections of entities. Each table contains rows, with each row representing a single instance of an entity.

Example:

- **Customer Table:** Each row contains data for a single customer.
- **Product Table:** Each row defines a single product.
- **Order Table:** Each row represents an order made by a customer.
- **Line Item Table:** Each row represents a product included in an order.

Columns and Data Types

- **Columns:** Define attributes of the entity. Each row in a table has the same columns, though not all columns need to have a value (can be NULL).

- **Data Types:** Each column stores data of a specific datatype.

Common Data Types:

- **Text Data:** Character-based (fixed or variable length) for columns like Email.
- **Numeric Data:** Decimal or integer values for columns like Price or Quantity.
- **Date/Time Data:** Date and time values for columns like OrderDate.

Standard Datatypes

- Most database systems support standard datatypes defined by the American National Standards Institute (ANSI).

Example Scenario

In a retail system:

- **Customer Table:** Contains customer details such as FirstName, LastName, Email, MiddleName.
- **Product Table:** Contains product details such as ProductID, ProductName, Price.
- **Order Table:** Contains order details such as OrderID, CustomerID, OrderDate.
- **Line Item Table:** Contains line items within an order such as OrderID, ProductID, Quantity.

Understand Normalization

Overview

Normalization is a schema design process that minimizes data duplication and enforces data integrity in a database.

Core Principles of Normalization

1. **Separate each entity into its own table.**
2. **Separate each discrete attribute into its own column.**
3. **Uniquely identify each entity instance (row) using a primary key.**
4. **Use foreign key columns to link related entities.**

Example Scenario

Un-normalized Data

- **Issues:**
 - Data duplication.
 - Combined customer and product details in the same cells.

Normalized Data

- **Changes:**
 - Each entity (e.g., customer, product, sales order, line item) is stored in its own table.
 - Each attribute is in its own column.
 - Primary keys uniquely identify rows.
 - Foreign keys link related entities.

Benefits of Normalization

- **Reduces Duplication:** Modifying a customer's address requires changing a single row.
- **Ensures Data Integrity:** Values are constrained to appropriate data types.
- **Granularity:** Allows detailed querying (e.g., filtering customers by city).
- **Referential Integrity:** RDBMS enforces that foreign key values exist in the related table (e.g., no orders for non-existent customers).

Composite Keys

- **Usage:** Unique combination of multiple columns to identify rows.
- **Example:** Lineltem table uses a combination of OrderNo and ItemNo to identify a line item in an order.

Explore SQL

Overview

SQL (Structured Query Language) is used to communicate with relational databases, allowing for tasks such as updating and retrieving data. Common relational database systems using SQL include Microsoft SQL Server, MySQL, PostgreSQL, MariaDB, and Oracle.

History

- **Standardization:**
 - ANSI (1986)
 - ISO (1987)
- **Extensions:** Most database vendors have proprietary extensions.

Common SQL Statements

Data Definition Language (DDL)

- **CREATE:** Create new objects in the database.
- **ALTER:** Modify existing objects.
- **DROP:** Remove objects.
- **RENAME:** Rename objects.

Data Control Language (DCL)

- **GRANT:** Grant permissions.
- **DENY:** Deny permissions.
- **REVOKE:** Remove previously granted permissions.

Data Manipulation Language (DML)

- **SELECT:** Read rows from a table.
- **INSERT:** Insert new rows.
- **UPDATE:** Modify existing rows.
- **DELETE:** Delete rows.

SQL Statement Types

Data Definition Language (DDL)

- Used to create, modify, and remove tables and other database objects.

Data Control Language (DCL)

- Used to manage access to objects in a database.

Data Manipulation Language (DML)

- Used to manipulate the rows in tables, including retrieval, insertion, modification, and deletion of data.

SQL Dialects

- **Transact-SQL (T-SQL):** Used by Microsoft SQL Server and Azure SQL services.
- **pgSQL:** Used by PostgreSQL.
- **PL/SQL:** Used by Oracle.

Describe Database Objects

Overview

In addition to tables, a relational database contains other structures to optimize data organization, encapsulate programmatic actions, and improve access speed. Three key structures are views, stored procedures, and indexes.

Views

- **Definition:** A virtual table based on the results of a SELECT query.
- **Usage:** Provides a simplified view of data from one or more tables, making it easier to work with complex queries.
- **Example:** A view combining order and customer data for easy retrieval of delivery addresses.

Stored Procedures

- **Definition:** Encapsulated SQL statements that can be executed on command.
- **Usage:** Encapsulates programmatic logic for actions that applications need to perform on data.
- **Flexibility:** Can be defined with parameters to perform actions based on specific criteria.
- **Example:** A stored procedure to change a product name based on product ID.

Indexes

- **Definition:** Helps search for data in a table efficiently.
- **Usage:** Similar to a book index, it creates a sorted set of references with pointers to the corresponding rows.
- **Efficiency:** Dramatically improves query performance, especially in large tables.
- **Considerations:**
 - Indexes consume storage space.
 - Maintenance of indexes can slow down insert, update, and delete operations.
 - Balance the number of indexes to optimize query speed versus operation performance.

Summary

- **Views:** Simplify data retrieval by creating virtual tables.
- **Stored Procedures:** Encapsulate SQL logic for repetitive tasks.
- **Indexes:** Enhance query performance but require careful management.

Azure SQL Services and Capabilities

Overview

Azure SQL is a family of Microsoft SQL Server-based database services in Azure. Key services include:

- **SQL Server on Azure Virtual Machines (VMs)**
- **Azure SQL Managed Instance**
- **Azure SQL Database**
- **Azure SQL Edge** (optimized for IoT scenarios)

SQL Server on Azure Virtual Machines (VMs)

- **Type:** IaaS
- **Compatibility:** Fully compatible with on-premises SQL Server installations.
- **Architecture:** SQL Server instances installed in a VM.
- **Availability:** 99.99%
- **Management:** Full control over server and database configuration.
- **Use Cases:**
 - Lift and shift migration of on-premises SQL Server.

- Hybrid deployments.
- Development and testing.
- **Business Benefits:**
 - Rapid development and test scenarios.
 - Lift-and-shift readiness.
 - Easy scaling of VM resources.

Azure SQL Managed Instance

- **Type:** PaaS
- **Compatibility:** Near-100% compatibility with on-premises SQL Server.
- **Architecture:** Multiple databases per instance, supports instance pools.
- **Availability:** 99.99%
- **Management:** Automated updates, backups, and recovery.
- **Use Cases:**
 - Cloud migration with minimal changes.
 - Systems using features like linked servers, Service Broker, Database Mail.
- **Business Benefits:**
 - Reduced administrative tasks.
 - Integration with Azure services (e.g., Azure Storage, Azure Event Hubs).
 - High compatibility with SQL Server Enterprise Edition.

Azure SQL Database

- **Type:** PaaS
- **Compatibility:** Supports most core SQL Server capabilities.
- **Architecture:** Single database or elastic pool.
- **Availability:** 99.995%
- **Management:** Fully automated updates, backups, and recovery.
- **Use Cases:**
 - New cloud solutions.
 - Applications with minimal instance-level dependencies.
- **Business Benefits:**
 - Low cost with minimal administration.
 - Automatic updates and patches.
 - Scalability and high availability.
 - Advanced threat protection and auditing.

Single Database

- **Description:** Quickly set up and run a single SQL Server database in the cloud.
- **Scalability:** Scale resources as needed.

Elastic Pool

- **Description:** Share resources across multiple databases.
- **Use Case:** Databases with varying resource requirements to reduce costs.

Business Benefits Across Services

- **SQL Server on VMs:** Full control, lift-and-shift, hybrid deployments.
- **Managed Instance:** Minimal admin tasks, high compatibility, integration with Azure services.

- **SQL Database:** Automatic updates, high availability, scalability, advanced security features.

Azure SQL Services for Open-Source Databases

Overview

Azure provides data services for popular relational database systems, including MySQL, MariaDB, and PostgreSQL. These services enable organizations to move to Azure without significant changes to their applications.

Open-Source Databases

MySQL

- **History:** Leading open-source relational database for LAMP stack apps.
- **Editions:** Community (free), Standard, and Enterprise.
- **Specialization:** Web applications, available for Linux and Windows.

MariaDB

- **History:** Created by the original developers of MySQL.
- **Specialization:** Performance optimization, Oracle Database compatibility, temporal data support.

PostgreSQL

- **Specialization:** Hybrid relational-object database, custom data types, extensible with code modules, geometric data support.
- **Query Language:** pgSQL, a variant of SQL with procedural features.

Azure Database for MySQL

- **Type:** PaaS implementation of MySQL Community Edition.
- **Features:**
 - High availability, scalability, and automatic backups.
 - Connection security with firewall rules and optional SSL.
 - Monitoring functionality with alerts, metrics, and logs.
- **Benefits:**
 - High availability and predictable performance.
 - Easy scaling and secure data.
 - Automatic backups and point-in-time restore.
 - Enterprise-level security and compliance.
 - Pay-as-you-go pricing.

Azure Database for MariaDB

- **Type:** PaaS implementation of MariaDB Community Edition.
- **Features:** Fully managed with built-in high availability and scaling.
- **Benefits:**
 - High availability and predictable performance.
 - Secure data protection.
 - Automatic backups and point-in-time restore.

- Enterprise-grade security and compliance.
- Pay-as-you-go pricing.

Azure Database for PostgreSQL

- **Type:** PaaS implementation of PostgreSQL.
- **Features:**
 - High availability, performance, scaling, and security.
 - Core set of frequently used extensions supported.
- **Benefits:**
 - High availability with built-in failure detection and failover.
 - Use of familiar pgAdmin tool for management.
 - Query store for monitoring and fine-tuning queries.
- **Flexible Server Deployment:**
 - Fully managed database service with high control and customization.
 - Cost optimization controls.

Explore Azure Blob Storage

Overview

Azure Blob Storage is a service that enables the storage of massive amounts of unstructured data as binary large objects (blobs) in the cloud. It is optimized for cloud-based storage, and applications can interact with it using the Azure Blob Storage API.

Containers and Blobs

- **Containers:** Group related blobs together; control read/write access at the container level.
- **Blobs:** Can be organized in a hierarchy of virtual folders.

Types of Blobs

1. **Block Blobs:**
 - Handled as a set of blocks, each up to 4000 MiB.
 - Maximum size of 190.7 TiB.
 - Suitable for large, discrete objects that change infrequently.
2. **Page Blobs:**
 - Organized as 512-byte pages.
 - Optimized for random read/write operations.
 - Maximum size of 8 TB.
 - Used for virtual disk storage for VMs.
3. **Append Blobs:**
 - Optimized for append operations.
 - Blocks added only to the end; no updates or deletions.
 - Maximum size of 195 GB.

Access Tiers

1. **Hot Tier:**
 - Default tier.
 - High-performance media for frequently accessed data.
2. **Cool Tier:**
 - Lower performance with reduced storage charges.

- For infrequently accessed data.
- Blobs can be migrated between Hot and Cool tiers.

3. Archive Tier:

- Lowest storage cost with increased latency.
- For historical data accessed rarely.
- Blobs are stored offline; retrieval takes hours and requires rehydration to Hot or Cool tier.

Lifecycle Management

- **Policies:** Automate the movement of blobs between tiers based on age and usage.
- **Deletion:** Policies can also delete outdated blobs.

Explore Azure Data Lake Storage Gen2

Overview

Azure Data Lake Storage Gen2 is the latest version of Azure Data Lake Store, integrated into Azure Storage. It combines the scalability and cost-control of blob storage with the hierarchical file system capabilities needed for big data analytics.

Key Features

- **Integration with Azure Storage:** Leverages the scalability and cost-control of storage tiers.
- **Hierarchical Namespace:** Provides a hierarchical file system for managing data.
- **Compatibility:** Works with major analytics systems like Hadoop in Azure HDInsight, Azure Databricks, and Azure Synapse Analytics.

Usage

- **Mounting:** Distributed file systems hosted in Azure Data Lake Storage Gen2 can be mounted by analytics systems to process large volumes of data.
- **Creation:** Enable the Hierarchical Namespace option when creating an Azure Storage account or upgrade an existing account.

Important Considerations

- **Upgrade Process:** Upgrading an existing Azure Storage account to support a hierarchical namespace is irreversible.

Explore Azure Files

Overview

Azure Files enables the creation of cloud-based network shares, similar to on-premises file shares, providing scalable and highly available cloud storage for files.

Key Features

- **Storage Account:** Azure Files is created within an Azure storage account.
- **Capacity:** Share up to 100 TB of data across multiple file shares, with a maximum single file size of 1 TB.
- **Concurrent Connections:** Supports up to 2000 concurrent connections per shared file.
- **Access:** Upload files via Azure portal, AzCopy utility, or synchronize using Azure File Sync.

Performance Tiers

1. **Standard Tier:** Uses hard disk-based hardware.

2. **Premium Tier:** Uses solid-state disks, offering greater throughput at a higher cost.

Protocols

- **Server Message Block (SMB):** Commonly used across Windows, Linux, and macOS.
- **Network File System (NFS):** Used by some Linux and macOS versions, requiring a premium tier storage account and virtual network configuration.

Benefits

- **Cost and Maintenance:** Eliminates hardware costs and maintenance overhead.
- **Scalability and Availability:** Provides scalable and highly available cloud storage for files.

Explore Azure Tables

Overview

Azure Table Storage is a NoSQL storage solution that stores key/value data items in tables. Each item is represented by a row containing columns for the data fields that need to be stored.

Key Features

- **NoSQL Storage:** Stores semi-structured data.
- **Unique Keys:** Each row has a unique key composed of a partition key and a row key.
- **Timestamp Column:** Records the date and time of data modification.

Differences from Relational Databases

- **No Foreign Keys or Relationships:** Lacks relational database features like foreign keys, relationships, stored procedures, and views.
- **Denormalized Data:** Each row holds the entire data for a logical entity, unlike relational databases where data is split across multiple tables.

Example

A table holding customer information might store:

- First name
- Last name
- One or more telephone numbers
- One or more addresses

The number of fields in each row can vary based on the data recorded for each customer.

Partitioning

- **Mechanism:** Groups related rows based on a common property or partition key.
- **Benefits:**
 - **Independent Partitions:** Can grow or shrink independently.
 - **Performance:** Narrow search criteria with partition keys improve performance by reducing I/O operations.
 - **Scalability:** Enhances scalability by organizing data effectively.

Keys

- **Partition Key:** Identifies the partition containing the row.
- **Row Key:** Unique to each row within the same partition.

- **Ordering:** Items in the same partition are stored in row key order, enabling efficient point and range queries.

Use Cases

- **Fast Access:** Quick retrieval of single rows or contiguous blocks of rows in a partition.
- **Scalable Storage:** Suitable for applications requiring scalable and flexible storage of semi-structured data.

Azure Cosmos DB

Overview

Azure Cosmos DB supports multiple NoSQL formats and provides APIs for various data stores, enabling developers to use familiar programming semantics to work with data. It abstracts the internal data structure, allowing seamless interaction through different APIs.

Key Features

- **APIs:** Supports multiple application programming interfaces (APIs) for different data stores.
- **Performance:** Uses indexes and partitioning for fast read and write operations, scalable to massive data volumes.
- **Multi-region Writes:** Allows globally distributed users to work with local replicas by adding Azure regions to the Cosmos DB account.
- **Scalability:** Automatically allocates space for partitions, each growing up to 10 GB.
- **Low Administrative Overhead:** Indexes are maintained automatically with minimal administrative tasks.

Use Cases

1. **IoT and Telematics:**
 - Ingests large amounts of data in bursts.
 - Integrates with Azure Machine Learning, Azure HDInsight, and Power BI.
 - Supports real-time processing using Azure Functions.
2. **Retail and Marketing:**
 - Used by Microsoft's e-commerce platforms.
 - Stores catalog data and supports event sourcing in order processing pipelines.
3. **Gaming:**
 - Essential for gaming applications requiring fast read/write operations.
 - Handles massive request spikes during game launches and updates.
 - Provides low-latency performance for an engaging in-game experience.
4. **Web and Mobile Applications:**
 - Suitable for modeling social interactions, integrating with third-party services, and building personalized experiences.
 - Supports rich iOS and Android applications using the Xamarin framework.

Azure Cosmos DB APIs

Overview

Azure Cosmos DB is a fully managed and serverless distributed database for both relational and non-relational workloads. It supports multiple open source database engines, enabling developers to use familiar programming semantics.

Supported APIs

Azure Cosmos DB for NoSQL

- **Format:** JSON document.
- **Query Language:** SQL-like syntax.
- **Example Use Case:** Storing and querying customer data using SQL queries.

Azure Cosmos DB for MongoDB

- **Format:** Binary JSON (BSON).
- **Query Language:** MongoDB Query Language (MQL).
- **Example Use Case:** Using MongoDB client libraries to work with products data.

Azure Cosmos DB for PostgreSQL

- **Format:** Relational tables.
- **Query Language:** SQL.
- **Scalability:** Automatically shards data to scale from single-node to multi-node setups.
- **Example Use Case:** Defining and querying a table of products.

Azure Cosmos DB for Table

- **Format:** Key-value tables.
- **Compatibility:** Similar to Azure Table Storage with enhanced scalability and performance.
- **Example Use Case:** Storing customer data and retrieving specific records using Table API.

Azure Cosmos DB for Apache Cassandra

- **Format:** Column-family storage structure.
- **Query Language:** SQL-like syntax.
- **Example Use Case:** Defining and querying an Employees table.

Azure Cosmos DB for Apache Gremlin

- **Format:** Graph structure with vertices (nodes) and edges (relationships).
- **Query Language:** Gremlin.
- **Example Use Case:** Managing and querying data in a graph structure, such as employee and department relationships.

Key Benefits

- **Scalability:** Automatically allocates space for partitions, enabling seamless scaling.
- **Multi-region Writes:** Supports globally distributed data with local replicas for fast access.
- **Minimal Administrative Overhead:** Automatically maintains indexes and handles partitioning.

Data Warehousing Architecture

Overview

Large-scale data analytics architecture typically includes the following elements:

Key Elements

Data Ingestion and Processing

- **Sources:** Data from transactional data stores, files, real-time streams, etc.
- **Processes:** ETL (Extract, Transform, Load) or ELT (Extract, Load, Transform) to clean, filter, and restructure data for analysis.
- **Transformation:** ETL transforms data before loading; ELT transforms data after loading.
- **Systems:** Distributed systems processing high volumes of data in parallel.
- **Types of Ingestion:** Batch processing for static data and real-time processing for streaming data.

Analytical Data Store

- **Types:**
 - Relational data warehouses
 - File-system based data lakes
 - Hybrid architectures (data lakehouses or lake databases)
- **Purpose:** Optimized storage for large-scale analytics.

Analytical Data Model

- **Purpose:** Pre-aggregates data for easier reporting, dashboards, and visualizations.
- **Structure:** Often represented as cubes aggregating numeric data across dimensions (e.g., total sales by product and region).
- **Functionality:** Supports "drill-up/drill-down" analysis by encapsulating relationships between data values and dimensions.

Data Visualization

- **Purpose:** Create reports, dashboards, and other visualizations.
- **Consumers:** Data analysts, data scientists, and non-technical users for self-service analysis.
- **Forms:** Printed reports, graphs, charts, web-based dashboards, interactive environments.
- **Insights:** Trends, comparisons, key performance indicators (KPIs).

Data Ingestion Pipelines

Overview

Data ingestion pipelines orchestrate ETL processes to ingest data into an analytical data store from various sources. Azure provides several tools to create and manage these pipelines.

Key Tools

- **Azure Data Factory:** Used to create and run data pipelines.
- **Azure Synapse Analytics:** Provides a unified workspace for managing all components of a data analytics solution.
- **Microsoft Fabric:** Another option for managing data pipelines in a unified workspace.

Pipeline Components

- **Activities:** Operations performed on data within the pipeline.
- **Input Dataset:** Source data for the pipeline.
- **Data Flow:** Sequence of activities that manipulate data incrementally.

- **Output Dataset:** Resulting data produced by the pipeline.

Integration

- **External Data Sources:** Pipelines can connect to and integrate with a variety of data services, allowing flexibility in data ingestion and processing.

Benefits

- **Scalability:** Handles large-scale data ingestion efficiently.
- **Flexibility:** Supports diverse data sources and services.
- **Unified Management:** Allows for centralized management of data analytics components.

Analytical Data Stores

Overview

Analytical data stores are optimized for data analytics rather than transactional workloads. There are two common types: data warehouses and data lakehouses.

Data Warehouses

- **Description:** Relational database optimized for data analytics.
- **Schema:** Often uses a star schema or snowflake schema.
 - **Star Schema:** Numeric values stored in central fact tables related to dimension tables (e.g., sales data aggregated by customer, product, store, and time).
 - **Snowflake Schema:** Extends star schema by adding tables related to dimension tables (e.g., product categories).
- **Usage:** Best for structured transactional data that can be organized into tables and queried using SQL.

Data Lakehouses

- **Description:** Combines features of data lakes and data warehouses.
- **Data Lake:**
 - **Storage:** File store on a distributed file system for high performance access.
 - **Schema-on-Read:** Defines schemas at the point of reading data, not when storing.
 - **Support:** Structured, semi-structured, and unstructured data.
- **Hybrid Approach:**
 - **Lake Database/Data Lakehouse:** Stores raw data as files in a data lake with a relational storage layer abstracting files as tables.
 - **SQL Pools in Azure Synapse Analytics:** Uses PolyBase to define external tables based on files in a data lake and query them using SQL.
 - **Lake Database in Synapse Analytics:** Uses database templates to define relational schema while storing data in a data lake.

Technologies:

- **Delta Lake:** Adds relational storage capabilities to Spark, supporting batch-loaded and streaming data sources, and providing a SQL API for querying.

Key Differences

- **Data Warehouses:**
 - Optimized for structured data and SQL queries.
 - Enforces schema at the time of writing data.
- **Data Lakehouses:**
 - Flexible for mixed data types (structured, semi-structured, unstructured).
 - Enforces schema at the time of reading data.

Use Cases

- **Data Warehouses:** Ideal for structured transactional data requiring complex SQL queries.
- **Data Lakehouses:** Suitable for diverse data types needing flexible schema and high-performance processing.

Analytical Platform-as-a-Service (PaaS) Solutions

Overview

On Azure, three main PaaS services can be used to implement a large-scale analytical store:

1. Azure Synapse Analytics

- **Description:** A unified, end-to-end solution for large-scale data analytics.
- **Features:**
 - Combines a SQL Server-based relational data warehouse with a flexible data lake and Apache Spark.
 - Supports log and telemetry analytics with Azure Synapse Data Explorer pools.
 - Includes built-in data pipelines for data ingestion and transformation.
 - Managed through Azure Synapse Studio, which allows creating interactive notebooks combining Spark code and markdown content.
- **Use Case:** Ideal for creating a single, unified analytics solution on Azure.

2. Azure Databricks

- **Description:** An Azure implementation of the popular Databricks platform.
- **Features:**
 - Built on Apache Spark with native SQL capabilities and workload-optimized Spark clusters.
 - Provides an interactive user interface and supports interactive notebooks.
 - Commonly used on multiple cloud platforms, making it suitable for multicloud environments or cloud-portable solutions.
- **Use Case:** Best for leveraging existing expertise with Databricks or operating in a multicloud environment.

3. Azure HDInsight

- **Description:** Supports multiple open-source data analytics cluster types.
- **Features:**
 - Suitable for solutions relying on multiple open-source frameworks.
 - Can be used to migrate existing on-premises Hadoop-based solutions to the cloud.
- **Use Case:** Appropriate when multiple open-source frameworks are needed or for migrating Hadoop-based solutions.

Considerations

- **Analytical Data Store:** Each service provides a schema and interface for querying data.
- **Data Storage:** Data is often stored in a data lake, with services used to process and query the data.
- **Combination of Services:** Solutions might combine these services in an ELT process, such as using HDInsight or Databricks for processing and Synapse Analytics for querying.

Microsoft Fabric

Overview

Microsoft Fabric simplifies scalable analytics by providing a unified software-as-a-service (SaaS) offering. It integrates various services into a single product, making it easier to set up, create, and manage analytics solutions.

Key Features

Unified SaaS Offering

- **Simplified Management:** Combines multiple services into a single, easy-to-understand product.
- **Ease of Use:** Reduces complexity and fragmentation, streamlining the process of creating and managing analytics solutions.

OneLake

- **Lake-Centric Architecture:** Provides a single, integrated environment for data professionals and business users to collaborate on data projects.
- **Single Logical Lake:** Combines storage locations across different regions and clouds without moving or duplicating data.
- **Data Formats:** Supports any file format, structured or unstructured.
 - For tabular data, Fabric uses the delta format when writing to OneLake.
 - All analytical engines in Fabric can read and treat delta files as tables, regardless of the writing engine.

Benefits

- **Integrated Environment:** Facilitates collaboration between data professionals and business users.
- **Scalability:** Handles scalable analytics efficiently without the need for multiple fragmented services.
- **Open Format Storage:** Ensures compatibility and ease of use across different analytical engines.

Batch and Stream Processing

Overview

Data processing converts raw data into meaningful information. There are two general methods of data processing: batch processing and stream processing.

Batch Processing

- **Definition:** Collects and stores multiple data records to process together in a single operation.
- **Triggers:** Can be based on a scheduled interval, amount of data, or specific events.
- **Example:** Credit card billing where all purchases in a month are processed together.
- **Advantages:**
 - Processes large volumes of data at convenient times.
 - Can run during off-peak hours.
- **Disadvantages:**
 - Time delay between data ingestion and results.
 - Entire input data must be ready before processing.

Stream Processing

- **Definition:** Monitors and processes data in real-time as new events occur.
- **Example:** Real-time traffic counting or stock market monitoring.
- **Advantages:**
 - Immediate processing of data.
 - Ideal for time-critical operations.
- **Disadvantages:**
 - Typically processes only recent data.
 - Best suited for simple calculations or aggregates.

Differences Between Batch and Stream Processing

- **Data Scope:**
 - Batch: Processes all data in the dataset.
 - Stream: Processes recent data or data within a rolling time window.
- **Data Size:**
 - Batch: Handles large datasets.
 - Stream: Handles individual records or micro batches.
- **Performance:**
 - Batch: Latency is usually a few hours.
 - Stream: Latency is in seconds or milliseconds.
- **Analysis:**
 - Batch: Suitable for complex analytics.
 - Stream: Suitable for simple response functions or aggregates.

Combining Batch and Stream Processing

- **Approach:**
 - Capture real-time data with stream processing for immediate analysis or visualization.
 - Store data for subsequent batch processing.

- **Use Case:** Stream processing for real-time dashboards and batch processing for historical analysis.
- **Architecture:**
 - Stream processing captures and filters real-time data.
 - Batch processing ingests data from various sources for historical analysis.
 - Combined results are stored in an analytical data store for comprehensive analysis.

Common Elements of Stream Processing Architecture Overview

Stream processing solutions have common elements, though specific technologies may vary. A high-level architecture for stream processing includes the following components:

General Architecture for Stream Processing

1. **Event Generation:** Data is generated by an event, such as a sensor signal, social media post, or log entry.
2. **Data Capture:** The generated data is captured in a streaming source, which can be a folder, database table, or a queue that ensures ordered and unique processing.
3. **Data Processing:** Event data is processed by a perpetual query that selects, projects, or aggregates data over time periods.
4. **Output (Sink):** Processed results are written to a file, database table, real-time dashboard, or another queue for further processing.

Real-time Analytics in Azure

Azure supports multiple technologies for real-time analytics of streaming data:

1. **Azure Stream Analytics:** A PaaS solution for defining streaming jobs, ingesting data, applying queries, and writing results to an output.
2. **Spark Structured Streaming:** An open-source library for developing complex streaming solutions on Apache Spark-based services.
3. **Azure Data Explorer:** A high-performance database and analytics service optimized for ingesting and querying batch or streaming data with a time-series element.

Sources for Stream Processing

Common services for ingesting data for stream processing on Azure include:

1. **Azure Event Hubs:** Manages queues of event data, ensuring in-order and single processing.
2. **Azure IoT Hub:** Optimized for managing event data from IoT devices.
3. **Azure Data Lake Store Gen 2:** Scalable storage service used for both batch and streaming data.
4. **Apache Kafka:** Open-source data ingestion solution, commonly used with Apache Spark (available via Azure HDInsight).

Sinks for Stream Processing

The output from stream processing is often sent to the following services:

1. **Azure Event Hubs:** Queues processed data for further downstream processing.
2. **Azure Data Lake Store Gen 2 or Azure Blob Storage:** Persists processed results as files.
3. **Azure SQL Database, Azure Synapse Analytics, or Azure Databricks:** Persists processed results in database tables for querying and analysis.
4. **Microsoft Power BI:** Generates real-time data visualizations in reports and dashboards.

Azure Stream Analytics

Overview

Azure Stream Analytics is a service for complex event processing and analysis of streaming data. It is used to:

1. **Ingest Data:** From inputs like Azure Event Hub, Azure IoT Hub, or Azure Storage blob container.
2. **Process Data:** Using queries to select, project, and aggregate data values.
3. **Write Results:** To outputs such as Azure Data Lake Gen 2, Azure SQL Database, Azure Synapse Analytics, Azure Functions, Azure Event Hub, Microsoft Power BI, or others.

Key Features

- **Perpetual Queries:** Runs continuously, processing new data as it arrives.
- **Query Language:** Uses SQL syntax for defining data processing queries.
- **Static Reference Data:** Incorporates data from multiple sources for lookups.

Usage Scenarios

Azure Stream Analytics is ideal when you need to continually capture data from a streaming source, filter or aggregate it, and send the results to a data store or downstream process for analysis and reporting.

Stream Analytics Jobs and Clusters

- **Stream Analytics Job:**
 - **Setup:** Create in an Azure subscription, configure inputs and outputs, define the processing query.
 - **Processing:** Shared tenant with other customers.
- **Stream Analysis Cluster:**
 - **Setup:** For complex or resource-intensive processes.
 - **Benefits:** Dedicated tenant, configurable scalability for optimal throughput and cost.

Apache Spark on Microsoft Azure Overview

Apache Spark is a distributed processing framework for large-scale data analytics. It is used for both batch and stream processing and can run code written in Python, Scala, or Java across multiple cluster nodes.

Services Using Spark on Azure

1. **Azure Synapse Analytics**
2. **Azure Databricks**
3. **Azure HDInsight**

Spark Structured Streaming

- **Description:** Library for processing streaming data using an API to ingest, process, and output results from perpetual streams.
- **Dataframe:** Encapsulates a table of data; used to read real-time data into a "boundless" dataframe.
- **Query:** Define queries to select, project, or aggregate data in temporal windows, resulting in another dataframe for analysis.
- **Use Case:** Ideal for real-time analytics in Spark-based data lakes or analytical data stores.

Delta Lake

- **Description:** Open-source storage layer that adds transactional consistency, schema enforcement, and other data warehousing features to data lake storage.
- **Features:** Unifies storage for streaming and batch data, supports relational tables in Spark.
- **Integration:** Supported by Spark runtimes in Azure Synapse Analytics and Azure Databricks.
- **Use Case:** Combines with Spark Structured Streaming to abstract batch and stream processed data in a data lake behind a relational schema for SQL-based querying and analysis.

Real-time Analytics in Microsoft Fabric

Overview

Microsoft Fabric supports real-time data analytics, including real-time data ingestion from multiple streaming sources.

Key Features

Eventstream

- **Function:** Captures real-time event data from streaming sources.
- **Destination:** Data can be persisted in a Lakehouse table or a KQL database.

Lakehouse Table

- **Aggregations and Filters:** Apply these to summarize captured data.
- **Use Case:** Ideal for processing and storing real-time data for further analysis.

KQL Database

- **Engine:** Based on the Data Explorer engine.
- **Real-time Analytics:** Perform real-time analytics on data in tables by running KQL queries.

Power BI Integration

- **Visualization:** Use Power BI to create real-time data visualizations from captured data in tables.

Power BI Tools and Workflow

Overview

Microsoft Power BI is a suite of tools and services for creating interactive data visualizations for business users. It supports complex data modeling, interactive reporting, and secure sharing.

Power BI Components

Power BI Desktop

- **Function:** A Windows application for importing data from various sources, creating data models, and building reports with interactive visualizations.

- **Workflow:**
 - Import data from diverse sources.
 - Combine and organize data into an analytics data model.
 - Create interactive reports.

Power BI Service

- **Function:** A cloud service for publishing, sharing, and interacting with reports.
- **Features:**
 - Publish reports from Power BI Desktop.
 - Perform basic data modeling and report editing.
 - Schedule data source refreshes.
 - Share reports with users.
 - Define dashboards and apps to combine related reports.

Power BI Mobile App

- **Function:** Allows users to consume reports, dashboards, and apps on mobile devices.

Workflow Summary

1. **Create:** Use Power BI Desktop to import data, create data models, and build reports.
2. **Publish:** Publish reports to the Power BI Service for sharing and interaction.
3. **Consume:** Users access reports through web browsers or the Power BI mobile app.

Power BI Core Concepts of Data Modeling

Overview

Analytical models structure data to support analysis. Models are based on related tables of data and define numeric values (measures) and entities (dimensions) for aggregation. The model forms a multidimensional structure known as a cube.

Tables and Schema

Dimension Tables

- **Purpose:** Represent entities by which you want to aggregate numeric measures.
- **Example:** Product, customer, and time.
- **Attributes:** Columns representing entity details (e.g., product names, customer addresses).

Fact Tables

- **Purpose:** Store numeric measures for recorded events.
- **Example:** Sales transactions including quantity sold and revenue.
- **Schema Types:**
 - **Star Schema:** Fact table related to one or more dimension tables.
 - **Snowflake Schema:** Dimension tables related to additional tables for more details.

Example Schema

- **Customer, Product, and Time dimensions related to Sales fact table.**

Attribute Hierarchies

- **Purpose:** Enable quick drill-up or drill-down to find aggregated values at different levels in a hierarchical dimension.
- **Example:**

- **Product Table:** Category > Product.
- **Customer Table:** City > Customer.
- **Time Table:** Year > Month > Day.

Analytical Modeling in Microsoft Power BI

- **Power BI Desktop:**
 - Import data from one or more sources.
 - Use the Model tab to define relationships, hierarchies, data types, display formats, and other properties.
- **Functionality:** Helps create a rich model for analysis and reporting.

Power BI for Data Visualization

Overview

After creating a model, you can generate data visualizations for inclusion in a report. Power BI offers a variety of built-in visualizations, which can be extended with custom and third-party options.

Common Data Visualizations

Tables and Text

- **Tables:** Useful for displaying numerous related values.
- **Text Cards:** Show important figures or metrics.

Bar and Column Charts

- **Usage:** Visually compare numeric values for discrete categories.

Line Charts

- **Usage:** Compare categorized values and examine trends, often over time.

Pie Charts

- **Usage:** Compare categorized values as proportions of a total.

Scatter Plots

- **Usage:** Compare two numeric measures to identify relationships or correlations.

Maps

- **Usage:** Visually compare values for different geographic areas or locations.

Interactive Reports in Power BI

- **Interactivity:** Visual elements in a report are automatically linked and provide interactivity.
- **Example:** Selecting a category in one visualization filters and highlights that category in other related visualizations.

Example Scenario:

- **Visualization:** A report showing sales data.
- **Interactivity:** Selecting the city "Seattle" filters all visualizations to reflect values for Seattle.